Architectural Analysis and Technical Specification for MemPool-IPC

A Hardware-Assisted Rust Messaging Framework for RISC-V Many-Core Architectures

Author: Ping Long, Chief Systems Architect, Lead Researcher, SiliconLanguage Foundry

Contact: LinkedIn | GitHub | ping.long@siliconlanguage.com

The emergence of many-core architectures such as MemPool and its successor, TeraPool, necessitates a fundamental shift in how inter-processor communication (IPC) is conceptualized and implemented at the system level. Traditional IPC mechanisms, often mediated by a heavyweight operating system kernel, are insufficient for architectures that feature up to 1024 RISC-V harts sharing a massive, software-managed L1 scratchpad memory (SPM). The objective of the mempool-ipc framework is to provide a bare-metal, ultra-performant, and hardware-assisted messaging interface written in Rust, leveraging the unique characteristics of the Tightly Coupled Data Memory (TCDM) and the low-latency crossbar interconnect inherent in the PULP (Parallel Ultra Low Power) platform ecosystem.[1, 2]

RISC-V ISA Specifications and Memory Consistency Models

The implementation of a high-performance messaging framework on RISC-V requires a rigorous adherence to the unprivileged and privileged architecture specifications. Central to this is the memory consistency model, which dictates how memory operations are observed across the many-core fabric.

Unprivileged Architecture and Key Extensions

The RISC-V Unprivileged ISA provides the standard base instructions and extensions required for high-level language implementation. For mempool-ipc, three specific extensions are paramount: the "A" Standard Extension for Atomic Instructions, the "Zawrs" extension for Wait-on-Reservation-Set, and the "Zihintntl" extension for Non-Temporal Locality hints.[5]

The "A" extension (Version 2.1) provides instructions that atomically read-modify-write memory, supporting synchronization between multiple harts.[5] These instructions are divided into Load-Reserved/Store-Conditional (LR/SC) pairs and Atomic Memory Operations (AMOs). In a many-core environment like MemPool, where 256 to 1024 cores may contend for the same synchronization variables, AMOs (e.g., `amoswap`, `amoadd`) are often preferred over LR/SC loops because they can be implemented at the memory controller or within the interconnect, reducing the window of contention and ensuring eventually successful completion.[5, 6]

The "Zawrs" extension (Wait-on-Reservation-Set) is a critical optimization for power-efficient polling loops.[7, 8] In a traditional spin-loop, a core continuously executes load instructions, consuming significant energy and generating unnecessary traffic on the memory bus. Zawrs introduces the `WRS.NTO` (Wait-on-Reservation-Set No Timeout) and `WRS.STO` (Wait-on-Reservation-Set Short Timeout) instructions. These instructions allow a hart to temporarily stall

execution in a low-power state until a store occurs to the address range registered by a preceding `LR` instruction.[7, 8] For mempool-ipc, this mechanism enables a receiver to enter a sleep state while waiting for a message, waking only when the producer updates the queue's head or tail pointer.[8]

The "Zihintntl" extension provides hints to the hardware regarding the temporal locality of memory accesses.[9] In the MemPool architecture, where data movement between the L1 TCDM and L2/DRAM is common, these hints help avoid cache pollution. The `NTL.P1` hint, for instance, indicates that the data does not exhibit temporal locality in the innermost level of private cache.[9] By using these hints during large message transfers, mempool-ipc can ensure that transient IPC data does not displace critical working sets from the limited L1 memory or instruction caches.[11, 12]

RISC-V Weak Memory Ordering (RVWMO)

RISC-V utilizes the RVWMO model, a variant of release consistency that allows for aggressive reordering of memory operations to improve performance while maintaining a tractable programming model.[11] Under RVWMO, memory instructions from the same hart appear to execute in order from that hart's perspective, but may be observed in a different order by other harts.[11] This necessitates the use of `FENCE` instructions or atomic ordering annotations (`aq` for acquire and `rl` for release) to enforce synchronization.[5, 13]

The mapping of high-level C11/C++11 atomic operations to RISC-V instructions is standardized to ensure cross-compiler compatibility and correctness.[11]

Table 1: RVWMO Mappings for Atomic Loads, Stores, and Fences

C/C++ Construct	RISC-V Machine Mapping	Note
`atomic_load(relaxed)`	`l{b|h|w|d}`	No ordering
`atomic_load(acquire)`	`l{b|h|w|d}; fence r,rw`	Leading read-to-read/write fence
`atomic_load(seq_cst)`	`fence rw,rw; l{b|h|w|d}; fence r,rw`	Full fence surrounding the load
`atomic_store(relaxed)`	`s{b|h|w|d}`	No ordering
`atomic_store(release)`	`fence rw,w; s{b|h|w|d}`	Preceding read/write-to-write fence

`atomic_store(seq_cst)`	`fence rw,w; s{b|h|w|d}; fence rw,rw`	Full fence surrounding the store
`atomic_thread_fence(acquire)`	`fence r,rw`	Standard acquire barrier
`atomic_thread_fence(release)`	`fence rw,w`	Standard release barrier
`atomic_thread_fence(acq_rel)`	`fence.tso`	TSO-style barrier (or separate fences)
`atomic_thread_fence(seq_cst)`	`fence rw,rw`	Full ordering barrier

[11]

For read-modify-write (RMW) operations, the "A" extension provides direct hardware support for 32-bit and 64-bit operands.[5, 15]

Table 2: RVWMO AMO Mappings for RMW Operations

Operation Type	RISC-V AMO Instruction	Ordering Annotation
`atomic_<op>(relaxed)`	`amo<op>.{w|d}`	None
`atomic_<op>(acquire)`	`amo<op>.{w|d}.aq`	`.aq` bit set
`atomic_<op>(release)`	`amo<op>.{w|d}.rl`	`.rl` bit set
`atomic_<op>(acq_rel)`	`amo<op>.{w|d}.aqrl`	Both bits set
`atomic_<op>(seq_cst)`	`amo<op>.{w|d}.aqrl`	Both bits set

[11]

In cases where direct AMO instructions are unavailable (e.g., `compare_exchange`), LR/SC loops must be used with specific ordering bits to satisfy the RVWMO axioms.[5, 15]

Privileged ISA and Physical Memory Attributes (PMAs)

The privileged architecture defines Physical Memory Attributes (PMAs) which characterize each region of the physical address space.[13, 14] These attributes are hard-wired or configured via Physical Memory Protection (PMP) units.[13] For IPC, the distinction between "Main Memory" and "I/O" regions is vital. Main memory regions support cacheability and relaxed ordering, whereas I/O regions often have side effects on read/write operations (non-idempotent) and require strict ordering.[13, 14]

The mempool-ipc framework must account for these PMAs when mapping its queues. Specifically, memory regions used for hardware-assisted queues must be marked as non-idempotent and non-cacheable to ensure that accesses are not elided or reordered by the microarchitecture's speculative execution units.[13, 14] This is particularly relevant for the shared L1 TCDM in MemPool, which behaves like main memory for general data but can be partitioned for synchronization primitives.[1, 2]

The MemPool and TeraPool Ecosystem

MemPool is a many-core cluster architecture designed at ETH Zurich and the University of Bologna, targeting energy-efficient, high-performance computing for image processing and wireless communications.[1] The baseline MemPool implements 256 RISC-V cores sharing a large L1 TCDM through a low-latency interconnect.[1]

Hardware Architecture and Interconnect

The MemPool architecture is hierarchical, scaling from a single tile to a full cluster.[2] A tile consists of 4 cores sharing 16 banks of TCDM.[2] These tiles are grouped into clusters of up to 256 cores (MemPool) or 1024 cores (TeraPool).[1, 6] The interconnect is a fully-connected crossbar within smaller units and a hierarchical crossbar for larger scales, ensuring a maximum latency of 5 cycles for any core to access any L1 memory bank.[2, 6]

This low latency is achieved through a "physical-aware" design and the elimination of traditional hardware-managed caches at the L1 level, replaced by the software-managed scratchpad.[2] This architecture places the burden of memory management and coherence on the software (and thus, the IPC framework), but provides unmatched determinism and efficiency.[2]

Software Runtime and C Headers

The MemPool ecosystem includes a bare-metal C runtime that provides the foundations for software execution.[1, 2] This runtime handles hart initialization, stack setup, and provides macros for accessing platform-specific registers and memory regions.[1, 19]

While specific header files are distributed within the `software/` and `runtime/` directories of the `pulp-platform/mempool` repository, the architectural definitions can be summarized based on the PULP conventions.[1, 20]

Raw Code Snippet: MemPool Architectural Definitions (C)

C/C++

```c
/* Derived from PULP-Platform runtime conventions for MemPool */
#ifndef MEMPOOL_RUNTIME_H
#define MEMPOOL_RUNTIME_H

#include <stdint.h>

/* TCDM Memory Map (Configurable in config/config.mk) */
#define TCDM_BASE_ADDR    0x10000000
#define TCDM_BANK_SIZE    0x1000  /* 4KB per bank */
#define TCDM_NUM_BANKS    1024
#define TCDM_SIZE         (TCDM_BANK_SIZE * TCDM_NUM_BANKS)

/* Core Identification */
static inline uint32_t mempool_get_core_id() {
    uint32_t id;
    /* RISC-V Standard CSR for Hart ID */
    asm volatile ("csrr %0, mhartid" : "=r" (id));
    return id;
}

/* TCDM Bank Address Calculation */
static inline void* mempool_get_bank_addr(uint32_t bank_id) {
    return (void*)(TCDM_BASE_ADDR + (bank_id * TCDM_BANK_SIZE));
}

/* Memory Barrier for RVWMO */
static inline void mempool_barrier() {
    asm volatile ("fence rw, rw" ::: "memory");
}

/* Non-Temporal Locality Hint Macros */
#define NTL_P1()   asm volatile ("add x0, x0, x2")
#define NTL_PALL() asm volatile ("add x0, x0, x3")

#endif /* MEMPOOL_RUNTIME_H */
```

Simulation with Banshee

The Banshee emulator is a functional, instruction-accurate simulator designed for the Snitch and MemPool many-core systems.[16, 17] It uses static binary translation via LLVM to achieve high performance while maintaining accuracy for the target RISC-V ISA.[16] Banshee is particularly useful for debugging IPC mechanisms due to its extensive tracing capabilities.[16, 17]

- **Logging:** Banshee supports multiple logging levels (`error`, `warn`, `info`, `debug`, `trace`) controlled by the `BANSHEE_LOG` environment variable.[16, 17]
- **Instruction Tracing:** The `--trace` flag enables per-hart instruction logging, including cycle counts, PC, and memory accesses.[16, 17] This output can be piped to `spike-dasm` for human-readable disassembly.[16, 17]
- **HTIF:** Banshee implements a Host-Target Interface (HTIF) to allow the simulated cores to interact with the host system for basic I/O (e.g., `printf`) and termination.[16, 17]
- **DRAM Preloading:** For large-scale many-core simulations, Banshee supports preloading the DRAM with binaries or data files using the `--file-paths` and `--mem-offsets` flags.[16, 17]

Rust Bare-Metal and Concurrency Documentation

Developing mempool-ipc in Rust requires a `no_std` environment, as there is no standard operating system or memory allocator provided in the bare-metal MemPool environment.[19] Rust's `core` library provides the necessary primitives for low-level concurrency and hardware interaction.[22, 23]

Idiomatic Concurrency Primitives

In a `no_std` context, Rust provides atomic types within `core::sync::atomic`.[19] These types are guaranteed to be atomic at the hardware level, provided the target architecture (in this case, RISC-V with the "A" extension) supports them.[5, 22]

- **Atomic Ordering:** The `Ordering` enum (Relaxed, Acquire, Release, AcqRel, SeqCst) allows the programmer to specify how memory accesses should be ordered around an atomic operation.[19]
- **Spin Loops:** The `core::hint::spin_loop()` function is used in busy-waiting loops.[19] In a bare-metal RISC-V context, this can be mapped to the `pause` hint or a custom Zawrs-based wait.[7, 22]

MMIO and Volatile Safety

Memory-Mapped I/O (MMIO) is the primary method for interacting with hardware peripherals and shared scratchpads in MemPool.[1] Rust mandates the use of volatile operations to prevent the compiler from optimizing away or reordering accesses to these memory regions.[20, 21]

- core::ptr::read_volatile **and** write_volatile**:** These functions perform a bitwise copy of a value to or from a pointer.[20, 21] They are considered externally observable events and are guaranteed not to be elided or reordered relative to other volatile operations.[20, 21]
- **Safety Considerations:** Volatile accesses are not atomic and do not provide inter-thread synchronization on their own.[23, 26] Furthermore, using volatile operations on compound types (structs) is discouraged as it can lead to multiple instruction emissions, which may be undesirable for hardware registers.[23] The recommended practice is to perform volatile operations on primitive integer types (`u8` through `u64`, and `usize`).[23]

Inline Assembly with `core::arch::asm!`

The `asm!` macro is the standard way to integrate handwritten assembly into Rust code.[24] For RISC-V, it allows the emission of specific instructions like those in the Zawrs or Zihintntl extensions that might not have a direct high-level mapping.[7, 9]

Raw Code Snippet: Rust Inline Assembly for Zawrs (WRS.NTO)

```rust
#![no_std]
use core::arch::asm;

/// Wait on a memory location to change using Zawrs WRS.NTO
#[inline(always)]
pub unsafe fn wrs_nto_wait(addr: *const usize) {
    // 1. Register the reservation set using LR
    asm!(
        "lr.w x0, ({0})",
        in(reg) addr,
        options(nostack)
    );

    // 2. Execute the WRS.NTO instruction
    // Opcode: SYSTEM (0x73), Funct3: 0, Rd: 0, Rs1: 0, Funct12: 0x00D
    asm!(
        ".insn s 0x73, 0, x0, 0x00D",
        options(nomem, nostack)
    );
}
```

```rust
/// Hint for non-temporal locality (NTL.PALL)
#[inline(always)]
pub fn ntl_pall_hint() {
    unsafe {
        asm!(
            "add x0, x0, x3",
            options(nomem, nostack, pure)
        );
    }
}
```

Foreign Function Interface (FFI) and Bindgen

To leverage the existing MemPool C runtime, Rust must interface with C through FFI.[22, 30] This is typically done using `extern "C"` blocks.[19] For large header files, the `bindgen` tool can automatically generate Rust bindings from C headers, though in a bare-metal environment, care must be taken to ensure that the generated types are `no_std` compatible and correctly represent the memory layout.[22, 30]

Prior Art and Reference Frameworks

The development of mempool-ipc can be informed by existing hardware-assisted OS frameworks and high-performance Rust channel libraries.

ChamelIoT: Hardware-Assisted OS Framework

ChamelIoT is a framework for reconfigurable IoT platforms that provides agnostic hardware acceleration for kernel services such as scheduling, thread management, and IPC.[33, 34] It targets RISC-V systems, specifically those using the Rocket core and the Rocket Custom Co-Processor Interface (RoCC).[33, 34]

- **Queue Mapping:** ChamelIoT implements multiple ready-queues in hardware as linked lists.[33, 34] This allows the hardware to manage thread insertion and removal with bounded worst-case execution time (WCET), enhancing determinism and real-time guarantees.[33, 34]
- **Hardware Architecture:** The co-processor consists of a Control Unit, Status Registers, and Node Arrays.[33, 34] The Node Array stores thread metadata (ID, state, priority) and is managed directly by the hardware logic.[33, 34]
- **Implications for mempool-ipc:** While MemPool uses a shared scratchpad instead of a dedicated co-processor for IPC, the ChamelIoT approach of mapping logical OS structures (queues) to hardware structures (interleaved memory banks or hardware FIFO controllers) is a powerful model for achieving low latency.[33, 35]

Rust Lock-Free Channel Libraries

Libraries like `crossbeam-channel` and `flume` provide the state-of-the-art for high-performance message passing in Rust.[26, 27]

- **API Structure:** Both libraries provide a `Sender<T>` / `Receiver<T>` API, supporting Multi-Producer Multi-Consumer (MPMC) patterns.[26, 28]This is achieved by allowing both ends to be cloned and shared.[28]
- **Performance:** These libraries minimize the use of locks, preferring atomic operations for synchronization.[26, 27] `flume` is noted for its lightweight synchronization and minimal dependencies, often outperforming `crossbeam-channel` in specific benchmarks.[27, 28]
- **Design for mempool-ipc:** The mempool-ipc framework should mimic this API while replacing the underlying synchronization (which typically uses OS-level futexes or spinning) with the RISC-V "A" and "Zawrs" extensions.[5, 7]

Architectural Specification for mempool-ipc

The mempool-ipc framework is designed to exploit the shared-L1, scratchpad-based architecture of the MemPool system.

TCDM Memory Mapping and Interleaving

The shared L1 TCDM is the primary medium for IPC.[1] To maximize throughput and minimize contention, the framework utilizes an interleaved memory layout.[33, 34]

Table 3: TCDM Banking and IPC Partitioning

Region Type	Mapping Strategy	Access Pattern
Local SPM	Core-private banks	Fast, non-contested local data
IPC Queues	Interleaved across banks	High-bandwidth message transfer
Sync Variables	Specific dedicated banks	AMO-heavy control flow

Data is interleaved at a word or cache-line level to ensure that concurrent accesses by different harts are distributed across different physical banks.[33, 34] This is particularly effective for the fully-connected crossbar, which can handle multiple non-conflicting requests in parallel.[2, 6]

IPC Channel Protocol

The mempool-ipc channel protocol follows a lock-free, circular buffer implementation.

1. **Header:** Contains the queue capacity, element size, and atomic `head` and `tail` pointers.[5, 15]
2. **Payload:** A contiguous array of message slots in the TCDM.[1, 35]
3. **Flow Control:**
    - **Producer:** Checks the `tail` and `head` pointers to ensure the queue is not full. Writes the message using volatile stores (possibly with `NTL.PALL` hints if the data is large) and then updates the `tail` pointer using an atomic store-release.[11, 15]
    - **Receiver:** Monitors the `tail` pointer. If it equals the `head`, the receiver uses the `WRS.NTO` instruction to enter a low-power wait state.[7, 8] Once a message is available, it reads the data and updates the `head` pointer using an atomic release.[11]

Integration and Tooling

The framework is compiled as a `no_std` Rust library and linked against the MemPool C runtime.[22, 30]

- **Toolchain:** Requires a RISC-V GCC or LLVM toolchain with support for the "A", "Zawrs", and "Zihintntl" extensions.[5, 7, 9]
- **Verification:** Verified using the Banshee simulator, utilizing the `--trace` and `--trace-retired-ops` features to ensure correct memory ordering and interconnect utilization.[16, 17]

Analysis of Technical Dependencies and Future Outlook

The success of the mempool-ipc framework is contingent upon several hardware and software factors that characterize the evolving RISC-V landscape.

Hardware Interconnect and Scalability

The 5-cycle latency of the MemPool crossbar is the fundamental enabler for this framework.[2, 6] However, as the architecture scales to TeraPool (1024 cores), the interconnect topology becomes more hierarchical.[2] Future iterations of mempool-ipc will need to be "topology-aware," preferring to map queues between cores within the same tile or group to maintain low latency.[2, 35]

Language Safety and Low-Level Control

Rust's memory safety model, while primarily focused on the heap and stack within a single address space, provides strong guarantees for IPC through its ownership system.[28] By defining `Sender` and `Receiver` as types that encapsulate the underlying pointers and atomic logic, the framework prevents common errors like double-freeing a message slot or concurrent writes to the same queue entry without synchronization.[36, 38]

Extension Adoption

The "Zawrs" and "Zihintntl" extensions represent the latest in RISC-V architectural evolution.[7, 9] Their integration into the framework ensures that mempool-ipc is not just performant, but also power-efficient and sensitive to the complexities of modern memory hierarchies.[8, 11] As these extensions become standard in more RISC-V

implementations (e.g., beyond MemPool), the framework's portability across different many-core RISC-V SoCs will increase.

In conclusion, the mempool-ipc framework represents a sophisticated synthesis of many-core hardware design and modern system programming paradigms. By leveraging the low-latency TCDM of the MemPool architecture and the advanced synchronization primitives of the RISC-V ISA, it provides a foundation for ultra-performant, bare-metal applications in the next generation of computing clusters.

Technical Appendix: Reference Material and Source Links

For the purposes of co-development in a NotebookLM environment, the following primary sources and direct links are provided:

- **MemPool Repository:**
  https://github.com/pulp-platform/mempool [1]
- **MemPool Architecture Overview (Slides):**
  https://pulp-platform.org/docs/lugano2023/MemPool_05_06_23.pdf [2]
- **Banshee Simulator Repository:**
  https://github.com/pulp-platform/banshee [17]
- **Banshee: A Fast LLVM-Based RISC-V Binary Translator (Paper):**
  https://pulp-platform.org/docs/Banshee_ICCAD_2021.pdf [16]
- **Zawrs (Wait-on-Reservation-Set) Extension Draft:**
  https://github.com/riscv/riscv-zawrs/blob/main/zawrs.adoc [7]

This architectural brain is intended to serve as a comprehensive reference for the low-level implementation of the mempool-ipc framework, ensuring that all design decisions are grounded in the verified specifications and proven prior art of the RISC-V ecosystem.

---

[1] PULP Platform, "MemPool: A scalable 256/1024-RISC-V-core system," GitHub Repository. [Online]. Available: https://github.com/pulp-platform/mempool

[2] PULP Platform, "Diving into MemPool: Scaling the Shared-Memory Cluster to 256 Cores," Presentation Slides, 2023. [Online]. Available: https://pulp-platform.org/docs/lugano2023/MemPool_05_06_23.pdf

[3] S. Riedel et al., "Massively parallel and versatile? MemPool: Scaling the shared-memory cluster," DATE 2024 Poster, 2024. [Online]. Available: https://pulp-platform.org/docs/date2024/DATE2024_poster_samuel_manycore.pdf

[4] RISC-V International, "The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture," RISC-V Ratified Specifications. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html

[5] RISC-V International, "RISC-V ISA Manual: 'A' Standard Extension for Atomic Instructions," GitHub Repository. [Online]. Available: https://github.com/riscv/riscv-isa-manual/blob/main/src/a-st-ext.adoc

[6] RISC-V International, "Ratified Extensions," RISC-V Tech Hub. [Online]. Available: https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154732/Ratified+Extensions

[7] RISC-V International, "'Zawrs' Extension for Wait-on-Reservation-Set instructions, Version 1.01," RISC-V Documentation. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/zawrs.html
[8] RISC-V International, "Wait-on-Reservation-Set (WRS) Extension," RISC-V Mailing List. [Online]. Available: https://lists.riscv.org/g/apps-tools-software/attachment/180/0/Wait-on-Reservation-Set%20(WRS).pdf
[9] RISC-V International, "Zihintntl Extension for Non-Temporal Locality Hints, Version 1.0," RISC-V Documentation. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/zihintntl.html
[10] RISC-V International, "Compressed Zihintntl Instructions with Zca Extension," GitHub Issue #2212, riscv/riscv-isa-manual. [Online]. Available: https://github.com/riscv/riscv-isa-manual/issues/2212
[11] RISC-V International, "RVWMO Memory Consistency Model," The RISC-V Instruction Set Manual, Volume I: User-Level ISA. [Online]. Available: https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/rvwmo.html
[12] RISC-V Non-ISA, "RISC-V ELF psABI Document: Atomic Operations," GitHub Repository. [Online]. Available: https://github.com/riscv-non-isa/riscv-elf-psabi-doc/blob/master/riscv-atomic.adoc
[13] RISC-V International, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture," GitHub Pages. [Online]. Available: https://riscv.github.io/riscv-isa-manual/snapshot/privileged/
[14] OpenHW Group, "CV64A6_MMU Documentation: Privileged RISC-V ISA," CVA6 Documentation. [Online]. Available: https://cva6.readthedocs.io/en/latest/06_cv64a6_mmu/riscv/priv.html
[15] RISC-V International, "Vector Memory Ordering," RISC-V Tech Vector Extension Mailing List. [Online]. Available: https://lists.riscv.org/g/tech-vector-ext/topic/vector_memory_ordering/76634916
[16] F. Schuiki et al., "Banshee: A Fast LLVM-Based RISC-V Binary Translator," PULP Platform ICCAD 2021. [Online]. Available: https://pulp-platform.org/docs/Banshee_ICCAD_2021.pdf
[17] PULP Platform, "Banshee Repository," GitHub. [Online]. Available: https://github.com/pulp-platform/banshee
[18] PULP Platform, "MemPool Releases," GitHub. [Online]. Available: https://github.com/pulp-platform/mempool/releases
[19] Rust Project Developers, "Crate riscv," Docs.rs. [Online]. Available: https://docs.rs/riscv
[20] Rust Project Developers, "Module core::ptr: write_volatile," Rust Standard Library. [Online]. Available: https://doc.rust-lang.org/core/ptr/fn.write_volatile.html
[21] Rust Project Developers, "Module core::ptr: read_volatile," Rust Standard Library. [Online]. Available: https://doc.rust-lang.org/core/ptr/fn.read_volatile.html
[22] Rust Project Developers, "Module core::ptr," Rust Standard Library. [Online]. Available: https://doc.rust-lang.org/core/ptr/index.html
[23] Rust-Clippy Contributors, "ptr::write_volatile and read_volatile are not well-defined on compound types," GitHub Issue #15529, rust-lang/rust-clippy. [Online]. Available: https://github.com/rust-lang/rust-clippy/issues/15529
[24] Rust By Practice Contributors, "Inline assembly," Rust By Practice. [Online]. Available: https://practice.course.rs/unsafe/inline-asm.html
[25] Rust Users Forum, "Using custom asm instructions - embedded," The Rust Programming Language Forum. [Online]. Available: https://users.rust-lang.org/t/using-custom-asm-instructions/134808
[26] Rust Project Developers, "Crate crossbeam-channel," Crates.io. [Online]. Available: https://crates.io/crates/crossbeam-channel
[27] Rust Project Developers, "Crate flume," Docs.rs. [Online]. Available: https://docs.rs/flume
[28] Leapcell, "Building Robust Concurrent Pipelines with Crossbeam and Flume Channels in Rust," Leapcell Blog. [Online]. Available: https://leapcell.io/blog/building-robust-concurrent-pipelines-with-crossbeam-and-flume-channels-in-rust
[29] PULP Platform, "PULP Platform Official GitHub," GitHub. [Online]. Available: https://github.com/pulp-platform

[30] PULP Platform, "memory_island: An interleaved high-throughput low-contention L2 scratchpad memory," GitHub. [Online]. Available: https://github.com/pulp-platform/memory_island
[31] PULP Platform, "Chimera Repository," GitHub. [Online]. Available: https://github.com/pulp-platform/chimera
[32] PULP Platform, "MemPool Pull Requests," GitHub. [Online]. Available: https://github.com/pulp-platform/mempool/pulls
[33] Universidade do Minho, "Agnostic Hardware-Accelerated Operating System for Low-End IoT," RepositóriUM. [Online]. Available: https://repositorium.uminho.pt/bitstreams/0772c72c-9a79-46c4-9821-d5a5858b0983/download
[34] R. M. A. Silva et al., "ChamelloT: a tightly- and loosely-coupled hardware-assisted OS framework for low-end IoT devices," ResearchGate. [Online]. Available: https://www.researchgate.net/publication/376684347_ChamelloT_a_tightly-_and_loosely-coupled_hardware-assisted_OS_framework_for_low-end_IoT_devices
[35] R. M. A. Silva et al., "Leveraging RISC-V to build an open-source (hardware) OS framework for reconfigurable IoT devices," CARRV 2021. [Online]. Available: https://carry.github.io/2021/papers/CARRV2021_paper_52_Silva.pdf
[36] RISC-V International, "ISA Specifications Archive," RISC-V Tech Hub. [Online]. Available: https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154899/RISC-V+Technical+Specifications+Archive

---

Copyright (c) 2026 SiliconLanguage Foundry. All rights reserved.